

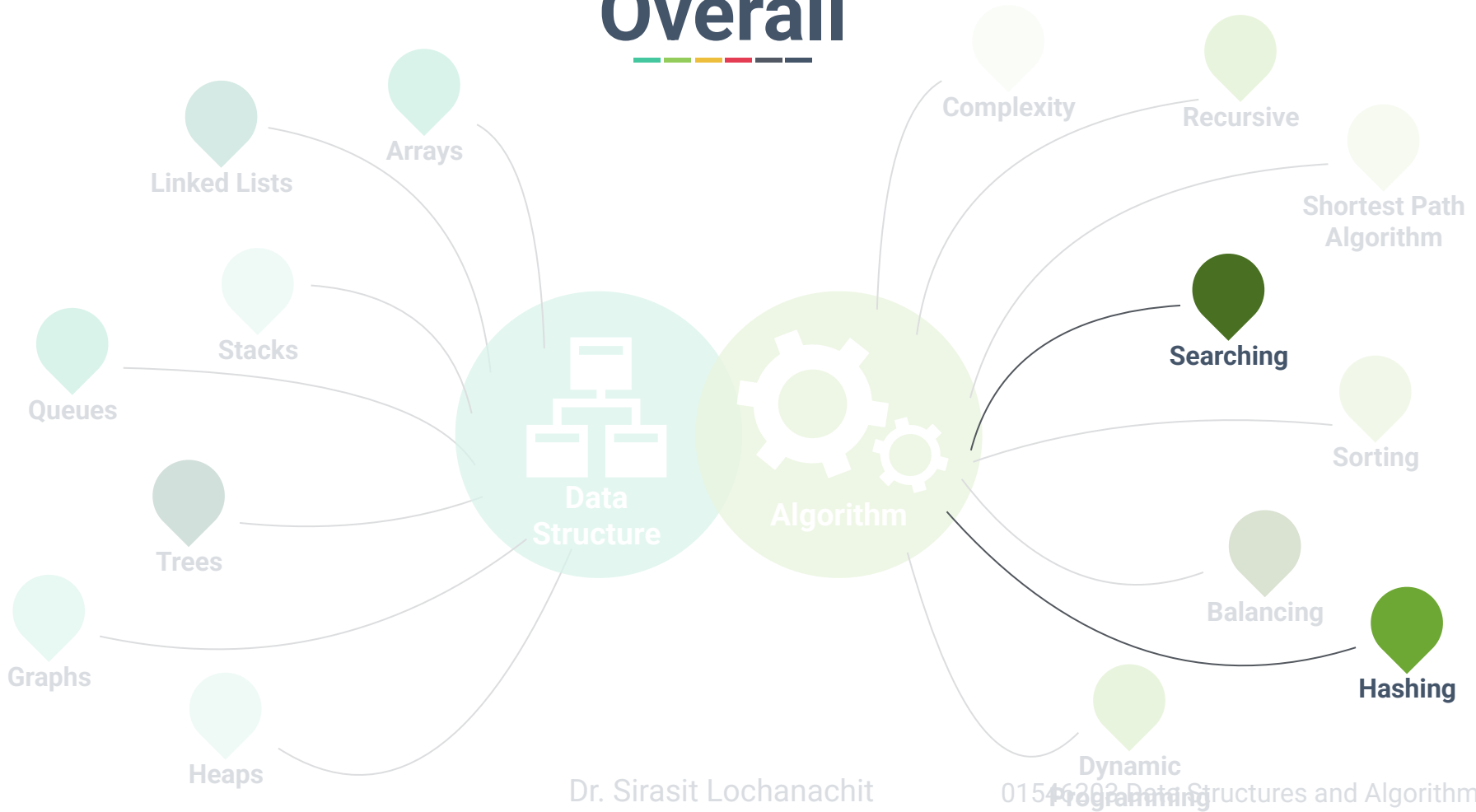
Week 3: Searching



Dr. Sirasit Lochanachit



Overall





Outline



Basic Search Algorithms:

- Sequential Search
- Binary Search
- Hashing:
 - Definition and method examples.
 - Example of collision resolutions



What is Searching?



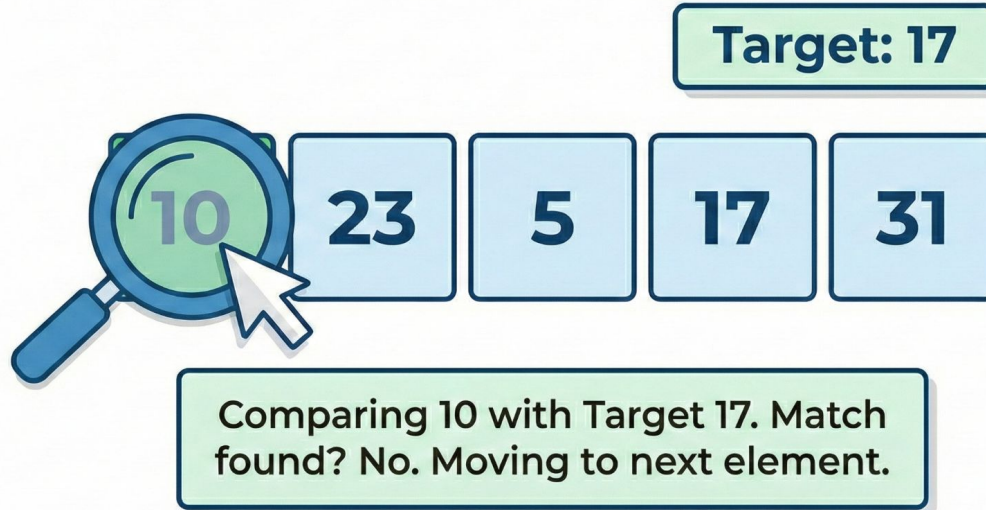
- The process of finding a particular item in a collection of items.
- Data management operations, such as insertion, updating, and deletion, usually requires searching algorithm to find an item as fast as possible.



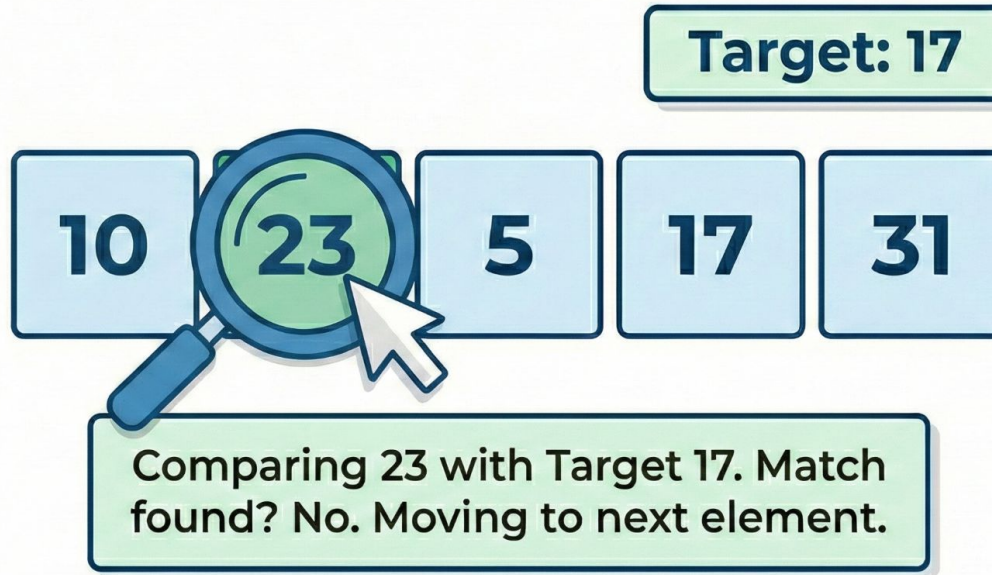
Sequential Search in an Array



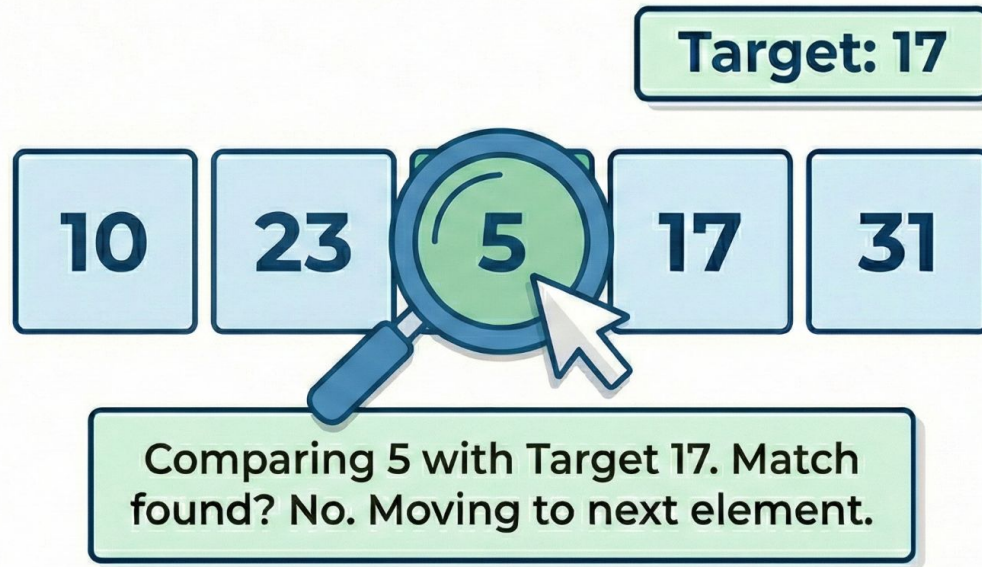
Sequential Search Algorithm - Step 1



Sequential Search Algorithm - Step 2



Sequential Search Algorithm - Step 3



Sequential Search Algorithm - Step 4





Sequential Search Algorithm



- Analysis of sequential search algorithm
 - Best Case
 - Worst Case
 - Average Case



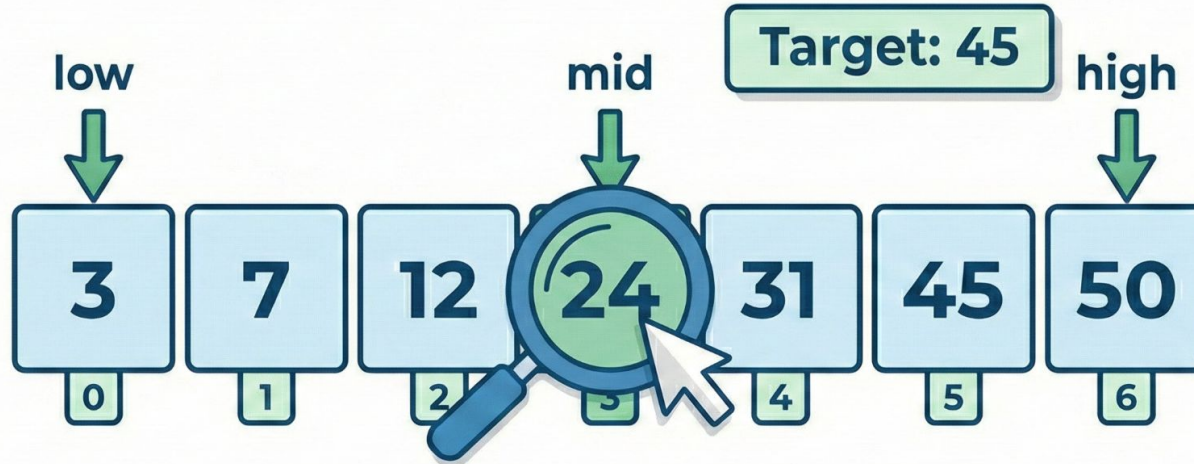
Binary Search



Locate a target value in a sequence of n elements that are **sorted**.

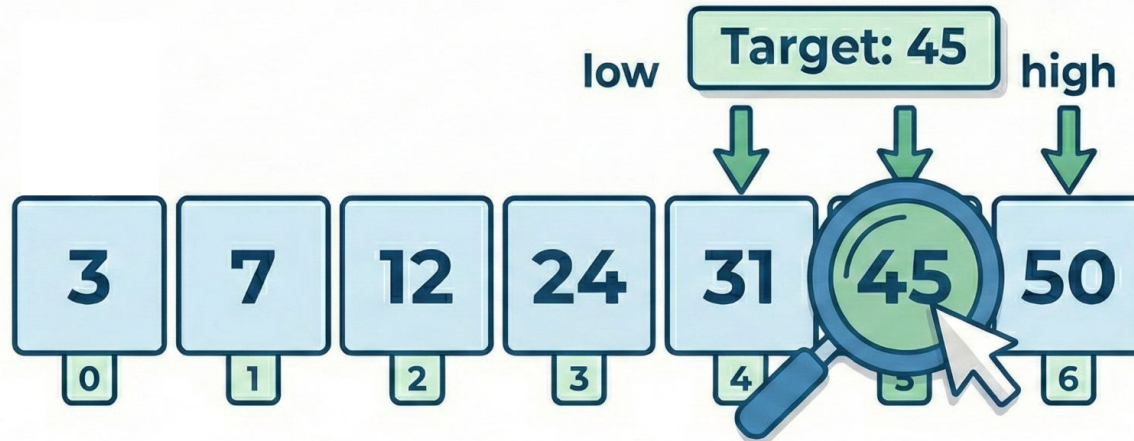
- low = 0th index
- high = $(n-1)$ th index
- mid = $((\text{low} + \text{high}) / 2)$ th index
 - mid is the main pointer to find a target value

Binary Search Algorithm - Step 1



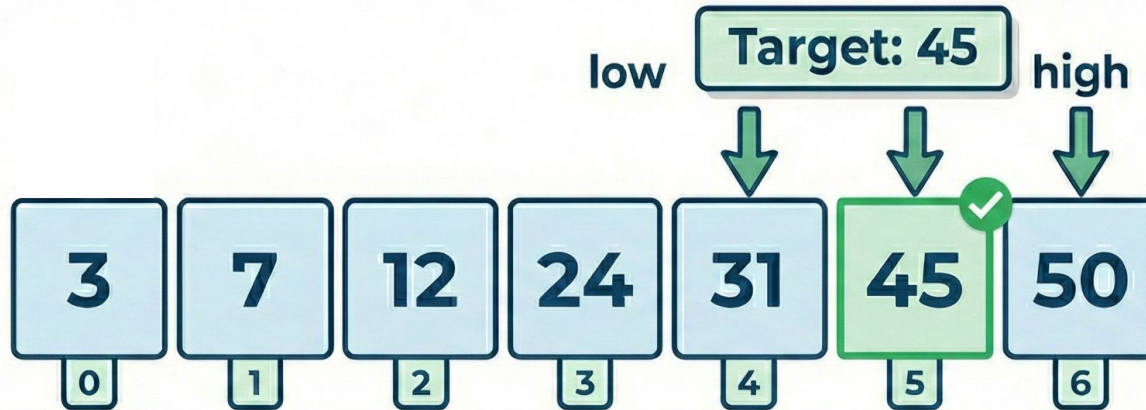
Initial range: [0, 6]. Mid index: 3.
Comparing 24 with Target 45.

Binary Search Algorithm - Step 2



Target (45) > Mid (24).
New range: [4, 6]. New mid index: 5.
Comparing 45 with Target 45.

Binary Search Algorithm - Step 3 (Match Found!)



Target (45) == Mid (45).
Match found! Search successful.
Returning index: 5.



Binary Search



- Implementation
 - If target value $< \text{data}[\text{mid}]$, next interval is from **low to mid-1**.
 - i.e. $\text{high} = \text{mid} - 1$
 - If target value $> \text{data}[\text{mid}]$, next interval is from **mid + 1 to high**.
 - i.e. $\text{low} = \text{mid} + 1$



Sequential and Binary Search



Sequential Search

- Easy to implement.
- Suitable for unsorted sequence or small array/list size.
- Search takes linear time - $O(n)$

Binary Search

- Suitable for a sorted sequence with large array/list size.
- Search takes logarithmic time - $O(\log n)$

What is Hashing?



Index

0	(Data)
1	(Data)
2	(Data)
3	(Data)
4	(Data)
5	(Data)

(1) Array

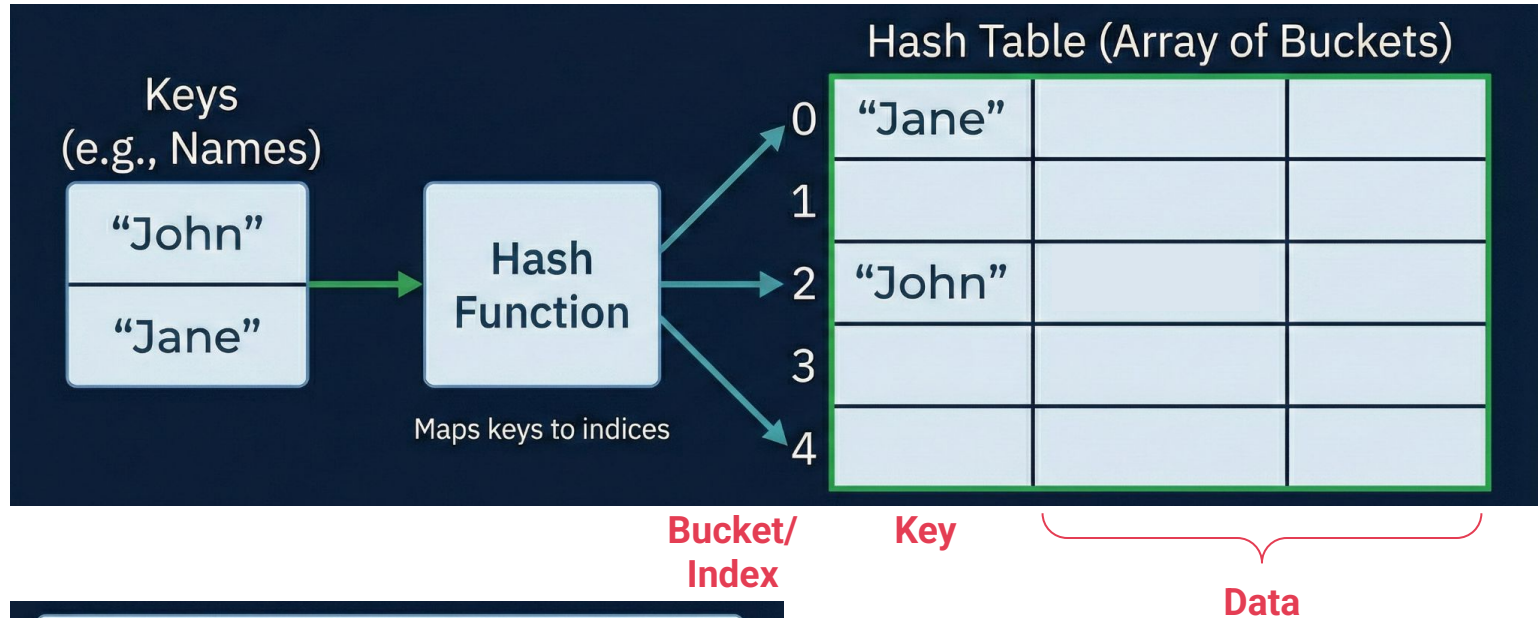
Bucket

	Hash Table	
0	(Key)	(Data)
1	(Key)	(Data)
2	(Key)	(Data)
3	(Key)	(Data)
4	(Key)	(Data)
5	(Key)	(Data)

(2) Hash table

Hashing is a technique that determines the index (bucket) using only a target search key, without the need to search.

What is Hashing?



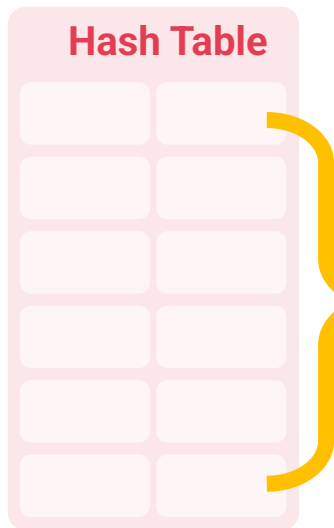
Data structure that implements an associative array, a structure that can map keys to values. A hash function computes an index into an array of buckets or slots.



Preparing Hashing Process



Step 1: define the capacity of the hash table (N).



N

Step 2: define the hash function.



Hashing Process



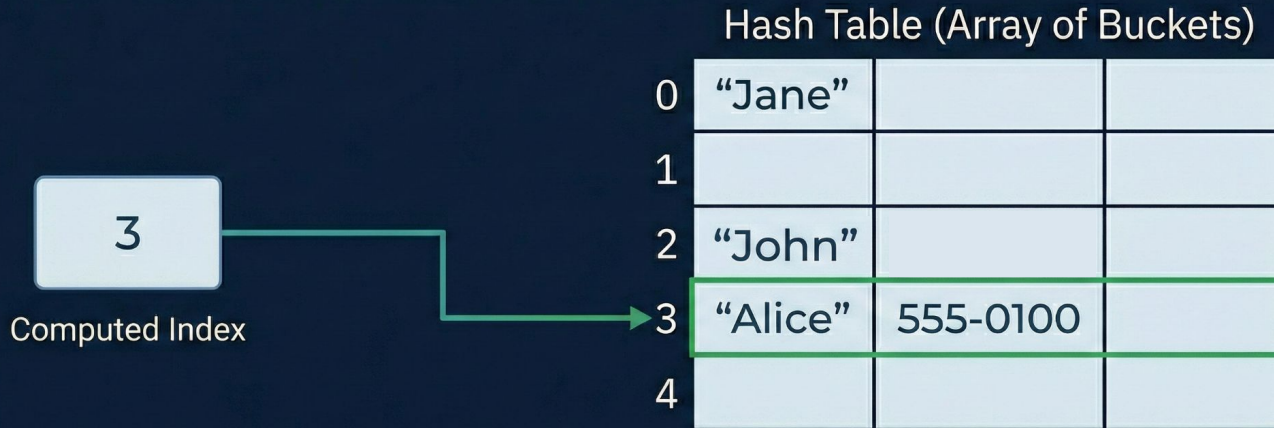
Hash Table Search - Step 1 (Hashing a Key)



The hash function takes the key "Alice" and calculates a unique index, in this case, 3.

Hashing Process

Hash Table Search - Step 2 (Looking up in the Table)



The computed index is used to directly access the corresponding bucket in the hash table. This is a direct lookup.

**Bucket/
Index**

Key

Data

Hashing Process



Hash Table Search - Search Complete



Search successful! Key 'Alice' found at index 3.
Retrieved Value: '555-0100'.
Average-case time complexity: $O(1)$.





Basic Hash Function



**Division method
(or modulo arithmetic)**

$$\text{Hash value} = k \bmod N$$

N = size of the bucket array

k = key of the item/element

Division Method



Bucket	Key/Data
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

$$\text{Hash value} = k \bmod N$$

Example:

Given a set of integer items {20, 19, 29, 10, 31, 3, 42, 14} and the capacity of the hash table is 11, calculate the hash values for each item.

Division Method

$$\text{Hash value} = k \bmod N$$

Bucket	Key/Data
0	
1	
2	
3	3, 14
4	
5	
6	
7	29
8	19
9	20, 31, 42
10	10

To search for an item:

- Simply use the hash function to compute the bucket index for the item.
- Then, check the hash table to see if it exists.

It takes a constant time **$O(1)$** to compute the hash value and retrieve the item.

Highly likely to cause collision of hash values.



Collision



Bucket	Key/Data
0	
1	
2	
3	3, 14
4	
5	
6	
7	29
8	19
9	20, 31, 42
10	10

$$\text{Hash value} = k \bmod N$$

If two or more keys produces the same hash value,

- They would need to be in the same bucket.
- This is known as **collision**.

Division Method Exercise 1



Bucket	Key/Data
0	
1	
2	
3	
4	
5	
6	
7	
8	

$$\text{Hash value} = k \bmod N$$

Exercise 1:

Given a set of integer items {4, 1, 3, 10, 7, 5, 6} and the capacity of the hash table is 9, calculate the hash values for each item.



Division Method Exercise 2

Bucket	Key/Data
0	
1	
2	
3	
4	
5	
6	
7	
8	

Dec	Hex	Char	Dec	Hex	Char
64	40	@	96	60	'
65	41	A	97	61	a
66	42	B	98	62	b
67	43	C	99	63	c
68	44	D	100	64	d
69	45	E	101	65	e
70	46	F	102	66	f
71	47	G	103	67	g
72	48	H	104	68	h
73	49	I	105	69	i
74	4A	J	106	6A	j
75	4B	K	107	6B	k
76	4C	L	108	6C	l
77	4D	M	109	6D	m
78	4E	N	110	6E	n
79	4F	O	111	6F	o
80	50	P	112	70	p
81	51	Q	113	71	q
82	52	R	114	72	r
83	53	S	115	73	s
84	54	T	116	74	t
85	55	U	117	75	u
86	56	V	118	76	v
87	57	W	119	77	w
88	58	X	120	78	x
89	59	Y	121	79	y
90	5A	Z	122	7A	z
91	5B	[	123	7B	{
92	5C	\	124	7C	
93	5D	]	125	7D	}
94	5E	^	126	7E	~
95	5F	_	127	7F	DEL

$$\text{Hash value} = k \bmod N$$

Exercise 2:

Given a set of character items {A, B, F, G, K, P, R, X, Y, Z} and the capacity of the hash table is set to 9, calculate the hash values for each item.



Collision Resolution



Two fundamental method for resolving collisions are:

- **First**, use another available spot in the hash table.
- **Second**, change the structure of the hash table so that each array element can store more than one value.



Collision Resolution



First, use another available spot in the hash table.

Linear Probing

Quadratic Probing

Double Hashing

Open addressing is the simple process to find the next empty bucket or address in the hash table.

It starts at the original hash value position and then move to the next hash value until the available bucket is found.



1. Open Addressing



Bucket	Key/Data
0	20
1	
2	
3	
4	
5	34
6	19
7	
8	
9	

Data = 10
Hash value = 0



Bucket	Key/Data
0	20
1	
2	
3	
4	
5	34
6	19
7	
8	
9	

10





Linear Probing

Linear probing method resolves a collision during hashing by looking at a sequential location, index by index, in the hash table that is available - starting at the original hash value. This search sequence is **probe sequence**.

Bucket	Key/Data
0	20
1	
2	
3	
4	
5	34
6	19
7	
8	
9	

Data = 10
Hash value = 0

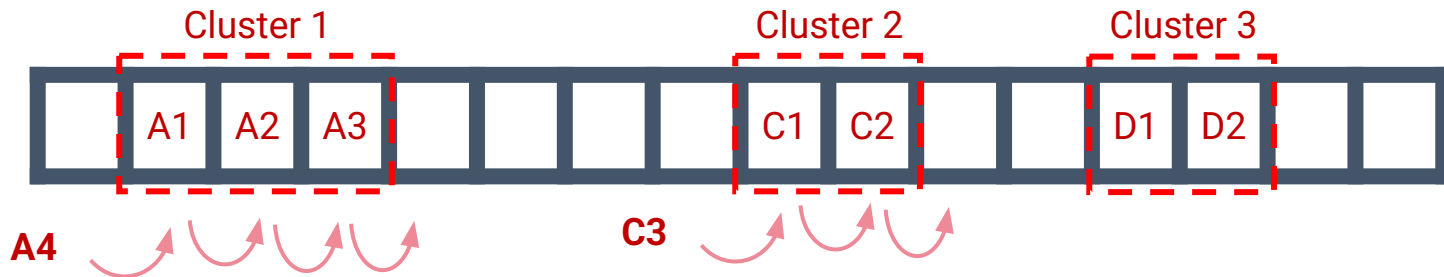


Bucket	Key/Data
0	20
1	10
2	
3	
4	
5	34
6	19
7	
8	
9	





Linear Probing



Although linear probing is a simple technique, it faces *a primary clustering problem*.

Collisions that are resolved with linear probing cause groups of consecutive locations (as so-called a cluster) in the hash table to be occupied.

2. Chaining

Second, Changing the structure of the hash table so that each array element can represent more than one value by using the **Linked List** structure.

